



UNIVERSIDAD SERGIO ARBOLEDA

**OPTIMIZACIÓN POR COLONIA DE HORMIGAS
PARA EL PROBLEMA DEL AGENTE VIAJERO:
20, 2 000 Y 200 000 CIUDADES**

INTELIGENCIA ARTIFICIAL

DOCENTE
JOAQUÍN FERNANDO SÁNCHEZ CIFUENTES

INTEGRANTES
GABRIELA DELGADO TIBAMOSO
JULIAN CAMILO CARO HERRERA
NICOLÁS STEVEN PÉREZ CHUSCANO

1 El punto de partida: tres tamaños que no son el mismo problema

La tarea pedía implementar ACO para el problema del agente viajero con 20, 2 000 y 200 000 ciudades, midiendo tiempo y memoria. Lo primero que hicimos, antes de programar, fue calcular cuánto ocuparían las estructuras del ACO que vimos en clase. Ese ACO guarda dos tablas de $n \times n$: la matriz de distancias D (cuánto hay de cada ciudad a cada otra) y la matriz de feromona T (qué tan “marcado” está cada camino). Cada casilla es un número de 8 bytes.

n	caminos $n(n-1)/2$	una matriz $n \times n$	qué implica
20	190	3.2 KB	se puede calcular la respuesta exacta
2 000	2.0×10^6	32 MB	funciona, pero no cabe en la caché
200 000	2.0×10^{10}	320 GB	no cabe en los 16 GB de RAM del equipo

Cuadro 1: Lo que ocuparía cada matriz del ACO de clase en los tres tamaños.

La tabla cambió el enfoque del trabajo. Con 200 000 ciudades, las dos matrices suman 640 GB, cuarenta veces la memoria del computador. Ese caso no se resuelve con un lenguaje más rápido ni con más paciencia: hay que cambiar la forma en que el programa representa el problema. Por eso organizamos todo alrededor de una pregunta: **¿cuánto se gana cambiando el diseño, y cuánto cambiando el lenguaje?**

Con 20 ciudades pasa lo contrario: el problema es tan pequeño que la respuesta exacta se puede calcular por programación dinámica (algoritmo de Held-Karp), en 0.31 segundos. Ese caso nos sirvió como verdad de referencia para saber si el ACO acertaba.

Tomamos además una decisión que el enunciado no pedía: medir también la *calidad* del recorrido. Si solo se mide tiempo y memoria, el mejor programa sería uno que devuelva cualquier orden de ciudades sin pensar.

2 El algoritmo que implementamos

2.1 Cómo elige una hormiga la siguiente ciudad

Cada hormiga arranca en una ciudad al azar y va armando su recorrido paso a paso. Estando en la ciudad i , a cada ciudad j que todavía no ha visitado le asigna la probabilidad

$$p_{ij} = \frac{\tau_{ij}^\alpha \eta_{ij}^\beta}{\sum_{l \text{ no visitada}} \tau_{il}^\alpha \eta_{il}^\beta}, \quad \eta_{ij} = \frac{1}{d_{ij}}. \quad (1)$$

En palabras simples

Es la ruleta vista en clase. A cada ciudad pendiente se le da un **puntaje**:

$$\text{puntaje} = \underbrace{\tau_{ij}}_{\text{feromona}} \times \left(\underbrace{\frac{1}{d_{ij}}}_{\text{cercanía}} \right)^2.$$

La feromona dice cuánto les ha servido ese camino a las hormigas anteriores; la cercanía premia las ciudades próximas, y el cuadrado ($\beta = 2$) la hace pesar más. La probabilidad de cada ciudad es su puntaje dividido por la suma de todos los puntajes (el denominador de (1) solo sirve para que las probabilidades sumen 1). Luego se gira la ruleta.

Ejemplo con tres ciudades pendientes:

ciudad	distancia	feromona	puntaje	probabilidad
A	4	2	$2 \cdot (1/4)^2 = 0.125$	$0.125/0.224 = 56\%$
B	6	3	$3 \cdot (1/6)^2 = 0.083$	37%
C	8	1	$1 \cdot (1/8)^2 = 0.016$	7%

2.2 Cómo aprende la colonia

Cuando las m hormigas terminan sus recorridos, la feromona de cada camino se actualiza:

$$\tau_{ij} \leftarrow (1 - \rho) \tau_{ij} + \sum_{k=1}^m \frac{Q}{L_k} [\text{la hormiga } k \text{ usó el camino } (i, j)]. \quad (2)$$

En palabras simples

Dos cosas pasan. Primero, toda la feromona se evapora un poco: con $\rho = 0.5$ cada camino pierde la mitad. Después, cada hormiga deja feromona en los caminos que usó, y deja más cuanto más corto fue su recorrido (Q/L_k : si su recorrido mide $L_k = 40$, deja $1/40$ en cada camino). Así los caminos de los buenos recorridos se vuelven más atractivos para la siguiente ronda. Una **iteración** es eso: las m hormigas construyen sus recorridos y luego se actualiza la feromona.

2.3 Los parámetros y por qué esos

- $\alpha = 1$, $\beta = 2$, $\rho = 0.5$, $Q = 1$: están dentro de los rangos que Dorigo y Stützle (los autores del método) recomiendan para esta variante ($\alpha = 1$, β entre 2 y 5, $\rho = 0.5$). De ese rango escogimos $\beta = 2$ por una razón práctica: con un exponente entero el puntaje se calcula multiplicando (**tau*eta*eta**) sin usar la función potencia. Eso nos permitió verificar que los cuatro lenguajes hacen exactamente las mismas cuentas (sección 5.1).
- $m = 10$ hormigas en los tres tamaños. Lo recomendado es una hormiga por ciudad, pero con 200 000 ciudades una sola iteración serían 200 000 recorridos. Con m fijo, el costo de una iteración es comparable entre tamaños.
- Feromona inicial $\tau_0 = m/L_{nn}$, donde L_{nn} es la longitud del recorrido del vecino más cercano (ir siempre a la ciudad más próxima).

Dejamos por fuera dos mejoras conocidas. La búsqueda local 2-opt, que toma el recorrido terminado y lo mejora, habría subido mucho la calidad, pero entonces estaríamos midiendo el 2-opt y no la colonia. La variante MAX-MIN también da mejores resultados, pero cambiarla junto con todo lo demás nos habría impedido saber qué produjo cada efecto.

3 Primer intento: la versión “normal”

Primero programamos el ACO tal como se ve en clase: matrices D y T completas, y en cada paso la ruleta sobre *todas* las ciudades pendientes. Antes de medirla calculamos cuánto iba a costar:

$$\text{memoria} = 2 \cdot 8n^2 \text{ bytes}, \quad \text{operaciones por hormiga} \approx \sum_{s=1}^{n-1} 2(n-s) \approx n^2.$$

En palabras simples

Una hormiga da n pasos, y en cada paso revisa todas las ciudades que le faltan. Eso son unas n^2 operaciones: **si se duplican las ciudades, el trabajo y la memoria se multiplican por 4**. Con 2 000 ciudades va bien; con 200 000 no hay computador personal que lo aguante.

4 La versión optimizada: qué cambiamos y por qué

La regla de la hormiga (ecuación 1) y la actualización (ecuación 2) son *exactamente las mismas*. Lo que cambió es sobre qué ciudades se aplica la ruleta y cómo se guardan los datos.

1. Solo los 10 vecinos más cercanos. Para cada ciudad guardamos una lista con sus $k = 10$ vecinas más cercanas, y la ruleta solo gira entre las que están en esa lista y no han sido visitadas. Cada paso pasa de revisar hasta n ciudades a revisar 10. Estas listas de candidatos las usan los propios autores del método en su implementación de referencia para el TSP; no son un agregado externo al ACO.

2. Feromona solo donde se usa. Si la ruleta solo mira esos 10 caminos por ciudad, solo hace falta guardar feromona para ellos: $10n$ números en vez de n^2 .

3. ¿Y si los 10 vecinos ya fueron visitados? Pasa en el 5.4 % de los pasos con 2 000 ciudades y en el 6.0 % con 200 000. En ese caso la hormiga va a la ciudad pendiente más cercana. Buscarla revisando todas las ciudades vuelve a costar n operaciones por paso, así que usamos una *rejilla*: dividimos el cuadrado en celdas de unas dos ciudades cada una y buscamos por anillos alrededor de la celda actual, del más cercano al más lejano.

El argumento para poder parar la búsqueda es geométrico. Si la celda de la hormiga tiene lado h y ya revisamos los anillos $0, 1, \dots, r$, cualquier ciudad de un anillo más externo está a una distancia de al menos $r \cdot h$, porque entre ellas hay por lo menos r celdas completas. Entonces, si la mejor ciudad encontrada está a una distancia $\leq r \cdot h$, ninguna ciudad por fuera puede ganarle y la búsqueda termina con la respuesta exacta.

4. Predicción de la memoria. Sumamos lo que ocupa cada estructura de la versión optimizada: coordenadas ($16n$ bytes), lista de vecinos ($4 \cdot 10n$), cercanías ($8 \cdot 10n$), feromona ($8 \cdot 10n$), los recorridos de las 10 hormigas ($4 \cdot 10n$) y la rejilla y auxiliares ($21n$). En total:

$$\text{memoria}_{\text{opt}} = 277n \text{ bytes.} \quad (3)$$

En palabras simples

La versión normal necesita $16n^2$ bytes y la optimizada $277n$. La primera crece con el *cuadrado* de las ciudades y la segunda de forma *proporcional*: si se duplican las ciudades, la normal ocupa 4 veces más y la optimizada 2 veces más. Con 200 000 ciudades la fórmula da unos 53 MB, contra 640 GB de la normal. Esta predicción la hicimos *antes* de medir; en la sección 6.2 la comparamos con lo medido.

5 Cómo lo medimos

5.1 Cuatro lenguajes haciendo exactamente las mismas cuentas

La versión optimizada la programamos en C++, Rust, Java y Python, y la normal en C++ y Python (con eso basta para separar el efecto del diseño del efecto del lenguaje; sección 6.3). No escogimos los lenguajes como “dos buenos y dos malos”: cada uno ejecuta el código de una forma distinta. C++ y Rust se compilan a código de máquina, Java se compila mientras corre (JIT) y tiene recolector de basura, y Python se interpreta línea a línea.

Para que la comparación midiera el *lenguaje* y no diferencias entre implementaciones, exigimos que las cuatro versiones hicieran exactamente las mismas cuentas. Para eso escribimos a mano el mismo generador de números aleatorios en los cuatro lenguajes (*splitmix64*), consumimos los aleatorios en el mismo orden y usamos solo operaciones que el estándar de punto flotante obliga a calcular igual en todas partes (suma, resta, multiplicación, división y raíz cuadrada). Si con la misma semilla los cuatro programas devuelven el mismo recorrido, están ejecutando el mismo algoritmo.

```
PS C:\Users\Yser\Downloads\IA_Pre_2026_01\taller-aco-tsp> bin\aco-cpp-gcc.exe opt 2000 2000 1 50 10 10 0 | python -c "import sys,json;r=json.load(sys.stdin);print('C++ ',r['best'],r['tour_hash'],r['t_busqueda'])"
C++ 40.1320384478 23f4a23526251e29 0.077353
PS C:\Users\Yser\Downloads\IA_Pre_2026_01\taller-aco-tsp> bin\aco-rust.exe opt 2000 2000 1 50 10 10 0 | python -c "import sys,json;r=json.load(sys.stdin);print('Rust ',r['best'],r['tour_hash'],r['t_busqueda'])"
Rust 40.1320384478 23f4a23526251e29 0.088237
PS C:\Users\Yser\Downloads\IA_Pre_2026_01\taller-aco-tsp> java -cp bin Aco opt 2000 2000 1 50 10 10 0 | python -c "import sys,json;r=json.load(sys.stdin);print('Java ',r['best'],r['tour_hash'],r['t_busqueda'])"
Java 40.1320384478 23f4a23526251e29 0.292058
PS C:\Users\Yser\Downloads\IA_Pre_2026_01\taller-aco-tsp> python python\aco.py opt 2000 2000 1 50 10 10 0 | python -c "import sys,json;r=json.load(sys.stdin);print('Python',r['best'],r['tour_hash'],r['t_busqueda'])"
Python 40.1320384478 23f4a23526251e29 5.012721
PS C:\Users\Yser\Downloads\IA_Pre_2026_01\taller-aco-tsp>
```

Figura 1: El mismo problema (2 000 ciudades, semilla 1, 50 iteraciones) en cuatro lenguajes. Columnas: longitud del mejor recorrido, código (*hash*) que identifica el recorrido, y segundos de búsqueda. Longitud y código coinciden; el tiempo no.

5.2 Tiempo y memoria

Cada programa, al terminar, imprime su resultado y se queda esperando. Mientras sigue abierto, un script aparte le pregunta a Windows cuál fue su máximo de memoria RAM (*memoria pico*). Lo hicimos desde fuera del programa para medir igual a los cuatro lenguajes, contando lo que pesan la máquina virtual de Java y el intérprete de Python, que también son costo del lenguaje.

En esta parte cometimos un error que vale la pena contar. En la primera medición, Java marcaba 10.3 MB con cualquier número de ciudades, incluso con 200 000, cuando solo los datos ya pasan de 50 MB. En Windows, el comando `java` es un lanzador que abre la máquina virtual como un proceso aparte, y nosotros estábamos midiendo el lanzador. Lo corregimos sumando el proceso y todos los que abre, y repetimos las corridas de Java.

El tiempo se midió dentro de cada programa, separando la *preparación* de la *búsqueda*. Todo corre en un solo núcleo, porque paralelizar depende mucho del lenguaje (Python casi no puede hacerlo) y habría distorsionado la comparación.

Equipo: AMD Ryzen 5 5600G (6 núcleos, caché L3 de 16 MB), 16 GB de RAM, Windows 11. Compiladores: g++ (MinGW-W64 x86_64-ucrt-posix-seh, built by Brecht Sanders, r3) 14.2.0; rustc 1.98.1 (48a229cea 2026-09-01); java version "22.0.2" 2024-07-16; Python 3.13.7. C++ y Rust se compilaban con optimización máxima (`-O3` y `opt-level=3`) para un procesador x86-64 genérico. Intentamos también `clang++`, pero no compiló en este equipo.

5.3 Con qué comparamos la calidad

Las ciudades son puntos al azar en un cuadrado de lado 1, generados siempre con la misma semilla para cada tamaño. Para juzgar qué tan buenos son los recorridos usamos tres referencias:

- $n = 20$: el recorrido óptimo exacto, calculado por Held-Karp: $L^* = 4.0758$.
- $n = 2000$ y $200\,000$: el óptimo no se puede calcular, pero para puntos al azar su valor esperado se conoce con buena precisión (ajuste de Cook et al.): $\mathbb{E}[L^*] \approx \sqrt{n} (0.712029 + 0.58816/\sqrt{n} + 0.568922/n)$, que da 32.44 y 319.0.
- **El vecino más cercano**: el método más ingenuo posible. Cualquier algoritmo que valga la pena tiene que superarlo.

La **brecha** de un recorrido es cuánto más largo es que la referencia: una brecha de 23.5 % significa un recorrido 23.5 % más largo que el óptimo.

6 Resultados

6.1 Los cuatro lenguajes dan el mismo recorrido

En las 25 combinaciones medidas (versión, tamaño y semilla), todos los lenguajes devolvieron el mismo recorrido con la misma longitud, hasta el último decimal (figura 1). Además, repetimos 70 de esas corridas en otro computador (Linux, procesador Intel) y los recorridos también coincidieron. Por eso, en lo que sigue, cualquier diferencia de tiempo entre lenguajes se debe al lenguaje y no a que hayan hecho cosas distintas.

6.2 Cómo crecen la memoria y el tiempo

Corrimos las dos versiones en C++ con tamaños crecientes, 5 iteraciones cada una. La versión normal la llevamos hasta 16 000 ciudades (4 GB); más allá ya no cabe en la RAM.

n	memoria pico (MB)		tiempo por iteración (ms)	
	normal	optimizada	normal	optimizada
500	8.0	3.9	3.3	0.4
1 000	19.8	4.0	15.3	0.9
2 000	67.5	4.3	61.2	1.7
4 000	248.6	4.8	234.0	4.3
8 000	981.7	5.9	938.3	7.5
16 000	3 913.6	8.0	4 726.0	16.1
50 000	—	17.0	—	59.8
100 000	—	30.2	—	160.2
200 000	—	56.8	—	402.3
500 000	—	135.9	—	1 188.6
1 000 000	—	267.9	—	2 559.3

Cuadro 2: Memoria pico y tiempo por iteración de las dos versiones en C++. “—”: la versión normal no se puede ejecutar a ese tamaño.

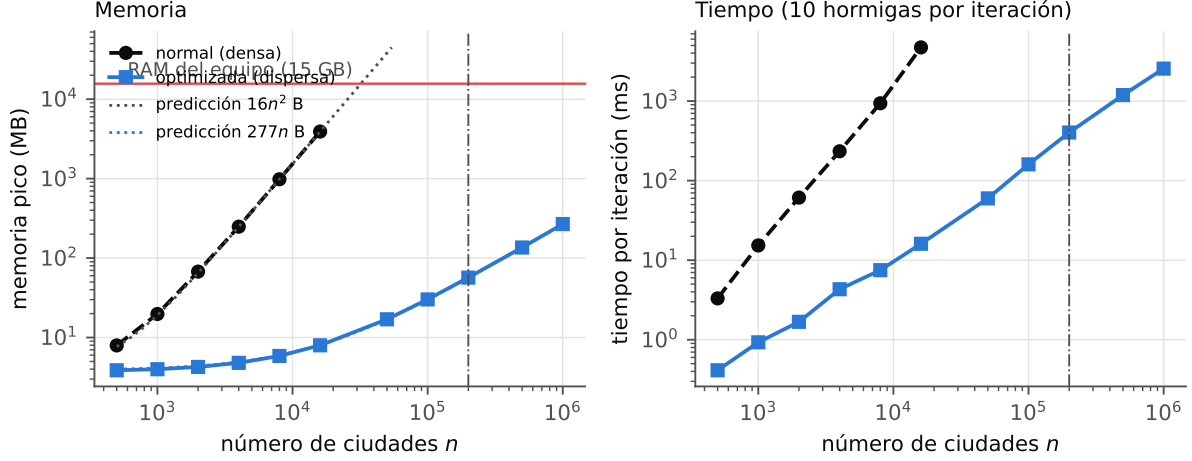


Figura 2: Los datos de la tabla 2 en gráfica. Líneas punteadas: nuestras predicciones hechas antes de medir. Línea roja: la RAM del equipo.

Cómo leer esto

Los dos ejes son **logarítmicos**: cada marca vale 10 veces la anterior. En esa escala, algo que crece con n^2 se ve como una recta con el doble de inclinación que algo que crece con n . Por eso la línea de la versión normal es más empinada.

El tramo plano al inicio de la versión optimizada *no* es una ventaja que luego se pierde: es la memoria fija que ocupa cualquier programa por el solo hecho de estar abierto (unos 3.8 MB). Con pocas ciudades, los datos pesan menos que eso y no se notan; la curva sube cuando los datos superan ese piso. En toda la tabla, la versión optimizada nunca crece más rápido que la normal.

La memoria resultó como la predijimos. La fórmula (3), más la memoria fija del programa, se equivocó como máximo en 0.1 MB entre 500 y 1000 000 ciudades. Con 200 000 ciudades predijimos 56.6 MB y medimos 56.8. Para la normal con 2000 predijimos 64.8 MB y medimos 67.5. Una forma de verlo en números: si se ajusta una recta a los puntos de la gráfica, su inclinación (pendiente) indica el exponente del crecimiento. Dio 1.98 para la normal y 1.00 para la optimizada; los valores teóricos eran 2 y 1.

El tiempo creció un poco más de lo previsto. Las pendientes del tiempo fueron 2.08 y 1.25, algo por encima de 2 y 1. Nuestra cuenta suponía que leer cualquier dato de la memoria cuesta lo mismo, y no es así: el procesador tiene una memoria pequeña y muy rápida (la caché L3, de 16 MB en este equipo). Mientras los datos caben ahí, todo va rápido; cuando no caben, cada lectura tiene que ir a la RAM, que es mucho más lenta. Por eso, a partir de cierto tamaño, cada operación cuesta más que al principio.

Lo que costaría la versión normal con 200 000 ciudades. Aunque existieran los 640 GB, cada iteración tardaría como mínimo 12 minutos. Lo calculamos a partir del tiempo medido con 16 000 ciudades, multiplicado por $(200\,000/16\,000)^2$. La versión optimizada hace esa misma iteración en 409 milisegundos, con 56.8 MB.

6.3 ¿Qué pesa más, el diseño o el lenguaje?

Para responderlo corrimos las dos versiones en el lenguaje más lento (Python) y en el más rápido (C++) con 2000 ciudades, y armamos una tabla de dos por dos.

	normal	optimizada	factor del diseño
<i>tiempo por iteración (ms), $n = 2000$</i>			
Python	3 855.6	102.78	38×
C++	60.6	1.54	39×
factor del lenguaje	64×	67×	2 503×
<i>memoria pico (MB), $n = 2000$</i>			
Python	322.4	13.2	24×
C++	67.5	4.3	16×
factor del lenguaje	4.8×	3.1×	

Cuadro 3: Diseño contra lenguaje con 2 000 ciudades. Comparando columnas se ve cuánto aporta el diseño; comparando filas, cuánto aporta el lenguaje.

Esperábamos que el diseño pesara más, y con 2 000 ciudades no fue así. Pasar de Python a C++ aceleró el programa 64 veces (versión normal) y 67 veces (versión optimizada). Pasar de la versión normal a la optimizada lo aceleró 38 veces en Python y 39 en C++. Las dos mejoras se multiplican: de Python normal a C++ optimizado hay una diferencia de 2 503 veces.

Esa conclusión cambia cuando crece el problema. La ventaja del diseño aumenta con el número de ciudades: la versión normal revisa n ciudades por paso y la optimizada 10, así que la diferencia es de $n/10$. La ventaja del lenguaje, en cambio, no crece; con 200 000 ciudades baja a 31 veces. La razón es que, a ese tamaño, C++ pasa buena parte del tiempo esperando datos de la RAM (el efecto de la caché de la sección anterior), mientras que Python es tan lento en cada operación que esa espera casi no se le nota. Con 200 000 ciudades, la versión normal no corre en ningún lenguaje, y Python con la versión optimizada sí resuelve el problema.

En memoria, el lenguaje pesa cuando la estructura es grande. Python guarda cada número de una lista como un objeto de 24 bytes más una referencia de 8, así que la versión normal ocupa en Python casi cinco veces lo que en C++. En la optimizada usamos `array('d')`, que guarda cada número en 8 bytes, y la diferencia se reduce a la memoria fija del intérprete.

6.4 Los cuatro lenguajes, uno por uno

n	versión	lenguaje	corridas	t/iter (ms)	preparación (ms)	pared (s)	memoria pico (MB)	brecha (%)
20	normal	Python	5	0.52	0.1	0.09	12.6	0.8
20	normal	C++ (g++)	5	0.01	0.0	0.01	3.8	0.8
20	opt	Python	5	0.73	0.4	0.11	12.6	0.3
20	opt	Java	5	0.09	0.2	0.13	52.0	0.3
20	opt	C++ (g++)	5	0.01	0.0	0.01	3.8	0.3
20	opt	Rust	5	0.01	0.0	0.01	4.6	0.3
2 000	normal	Python	1	3 855.58	638.4	193.46	322.4	73.4
2 000	normal	C++ (g++)	5	60.62	31.4	3.07	67.5	74.0
2 000	opt	Python	3	102.78	69.5	5.25	13.2	23.5
2 000	opt	Java	5	3.31	13.8	0.29	66.6	23.5
2 000	opt	C++ (g++)	5	1.54	1.7	0.09	4.3	23.5
2 000	opt	Rust	5	1.81	1.6	0.10	5.1	23.5
200 000	normal	Python	–	no ejecutable: requiere 640 GB				
200 000	normal	C++ (g++)	–	no ejecutable: requiere 640 GB				
200 000	opt	Python	1	12 813.40	8 493.0	136.90	82.1	30.4
200 000	opt	Java	5	633.71	435.7	6.89	134.1	30.5
200 000	opt	C++ (g++)	5	409.47	214.3	4.32	56.8	30.5
200 000	opt	Rust	5	505.49	214.7	5.28	58.2	30.5

Cuadro 4: Experimento principal (mediana de las corridas). Iteraciones: 100 ($n = 20$), 50 ($n = 2\,000$) y 10 ($n = 200\,000$).

Cómo leer esto

corridas: cuántas veces se ejecutó esa combinación, cada vez con otra semilla aleatoria para las hormigas. Python tiene menos porque cada corrida tarda minutos.

t/iter : tiempo de una iteración (las 10 hormigas más la actualización de feromona).

preparación: lo que hace el programa antes de soltar a las hormigas: llenar las matrices en la normal; buscar los 10 vecinos de cada ciudad en la optimizada.

pared: tiempo total desde que se lanza el programa hasta que responde, incluido lo que tarda en arrancar Java o Python.

memoria pico: la mayor cantidad de RAM que usó el programa.

brecha: cuánto más largo es el recorrido que la referencia. Con las mismas semillas es idéntica en todos los lenguajes, porque los recorridos son idénticos.

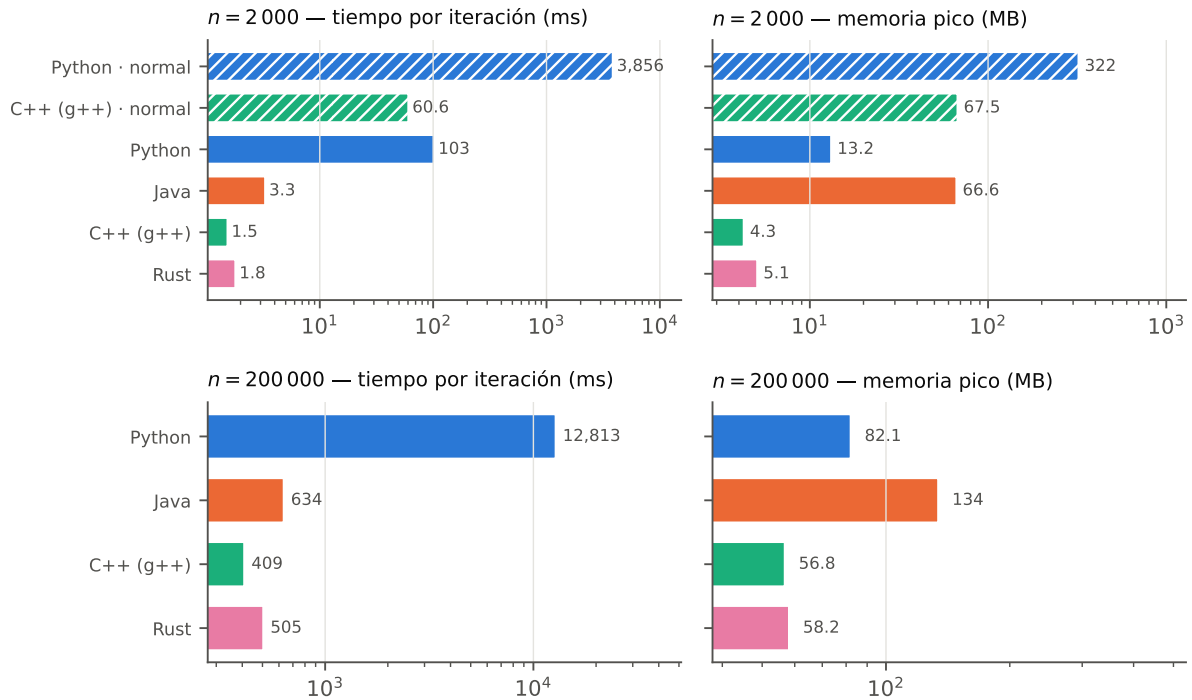


Figura 3: Tiempo por iteración y memoria pico de cada lenguaje. Las barras rayadas son la versión normal. Escala logarítmica: cada marca vale 10 veces la anterior, así que barras de longitud parecida pueden diferir varias veces en valor; los números al final de cada barra son los valores reales.

- **C++** fue el más rápido en todos los tamaños y, junto con Rust, el que menos memoria usó (56.8 MB con 200 000 ciudades).
- **Rust** quedó muy cerca: tarda 1.23 veces lo que C++ con 200 000 ciudades, con prácticamente la misma memoria (58.2 MB). Parte de esa diferencia viene de que Rust revisa, en cada acceso a un arreglo, que el índice esté dentro de los límites; C++ no lo hace. Como no pudimos usar `clang++`, que usa el mismo optimizador que Rust, no podemos separar cuánto se debe a eso y cuánto al compilador.
- **Java** es rápido (1.55 veces C++ con 200 000 ciudades), pero gasta más memoria desde el principio: su máquina virtual ocupa 52.0 MB antes de hacer nada, y con 200 000 ciudades llega a 134.1 MB, más del doble que C++. Con 20 ciudades, casi todo su tiempo total (128 ms, contra 9 ms de C++) es arranque de la máquina virtual.
- **Python** es entre 31 y 67 veces más lento que C++ haciendo exactamente las mismas cuentas, porque cada operación pasa por el intérprete. Aun así, con la versión optimizada resolvió el caso de 200 000 ciudades con 82.1 MB.

6.5 La optimización también mejoró los recorridos

Esto no lo esperábamos. Con 2 000 ciudades, la versión normal encuentra recorridos *peores que el vecino más cercano*, el método más ingenuo:

método ($n = 2000$)	longitud	brecha
ACO versión normal	56.46	74.0 %
vecino más cercano	42.28	30.3 %
ACO versión optimizada	40.06	23.5 %

La explicación está en la ruleta con $\beta = 2$. Imaginemos a la hormiga en una ciudad, con todas las demás repartidas al azar a su alrededor. Las ciudades a una distancia entre r y $r + dr$ forman un anillo cuya cantidad de ciudades es proporcional a su perímetro, $2\pi r dr$, y cada una tiene un puntaje proporcional a $1/r^2$. El peso total del anillo es entonces

$$\underbrace{2\pi r dr}_{\text{cuántas hay}} \times \underbrace{\frac{1}{r^2}}_{\text{puntaje de cada una}} = \frac{2\pi dr}{r}.$$

Sumando anillos, la probabilidad de saltar a una distancia entre a y b resulta proporcional a $\ln(b/a)$. Eso quiere decir que las distancias entre 0.01 y 0.1 tienen la misma probabilidad total que las distancias entre 0.1 y 1. Las ciudades lejanas tienen puntaje bajo, pero son tantas que juntas pesan tanto como las cercanas. Y cuantas más ciudades hay, más escalas de distancia existen, así que la probabilidad de quedarse cerca va bajando. Lo comprobamos midiendo, en el primer paso, qué fracción de la probabilidad cae en las 10 ciudades más cercanas:

ciudades	20	200	2 000	20 000	200 000
probabilidad de elegir una de las 10 más cercanas	0.88	0.66	0.50	0.44	0.35

En palabras simples

Con 2 000 ciudades, en la mitad de sus pasos la hormiga de la versión normal salta lejos de su vecindario, y cada salto largo le agrega al recorrido un tramo enorme. Limitar la ruleta a los 10 vecinos elimina esos saltos. No fue solo un ahorro de recursos: también corrigió un problema de calidad de la regla a esta escala.

Hay que decir que esto depende de haber usado $\beta = 2$. Con β mayor que 2 el puntaje de las ciudades lejanas cae más rápido que lo que crece su número, y la probabilidad sí se concentra cerca. Con otro β , la versión normal habría sufrido menos, así que no afirmamos que la versión densa sea peor en calidad en general.

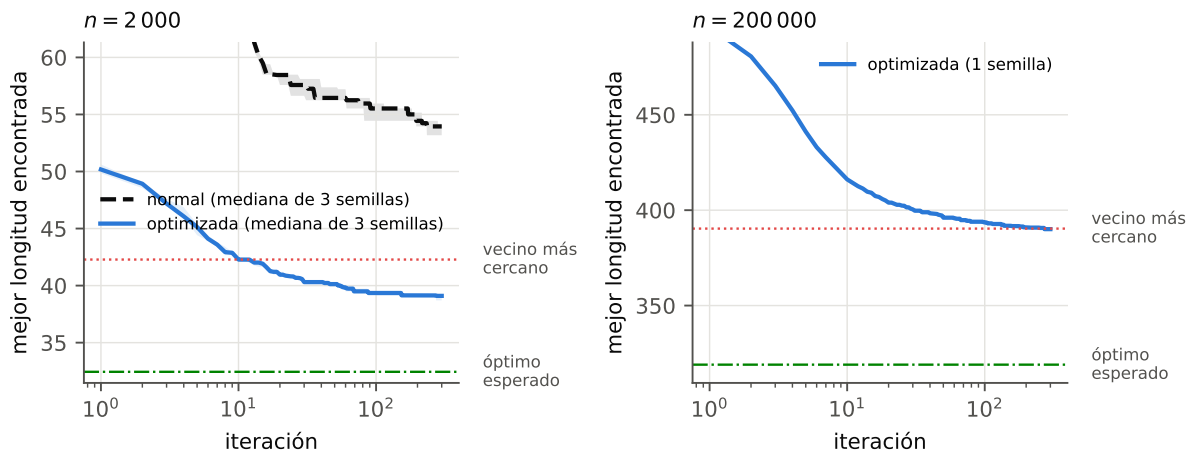


Figura 4: Longitud del mejor recorrido encontrado a lo largo de las iteraciones (C++).

Cómo leer esto

Cada línea muestra el mejor recorrido encontrado hasta esa iteración: **más abajo es mejor**. La línea roja punteada es el vecino más cercano y la verde es el óptimo esperado. El eje horizontal es logarítmico (1, 10, 100 iteraciones). A la izquierda (2000 ciudades), la versión normal (negra) nunca baja de la línea roja, y la optimizada (azul) la cruza en la iteración 10. A la derecha (200 000) solo aparece la optimizada, porque la normal no se puede ejecutar.

Seguir iterando no cierra la diferencia: tras 300 iteraciones, la normal queda en 53.95 y la optimizada en 39.10. Con 20 ciudades, en cambio, el efecto casi no existe (el 88 % de la probabilidad ya cae en los vecinos) y las dos versiones quedan a menos del 1 % del óptimo exacto: la optimizada lo encontró exacto en 3/5 semillas y la normal en 2/5.

6.6 Lo que el ACO no logra con 200 000 ciudades

Que el caso de 200 000 ciudades se pueda ejecutar no quiere decir que el ACO sirva a esa escala. Con 10 iteraciones, el mejor recorrido mide 416.2 (30.5 % de brecha), peor que el vecino más cercano (390.3, 22.4 %). En una corrida larga, la colonia recién lo iguala en la iteración 271 y termina en 390.1 tras 300 iteraciones (114 segundos). La razón es de escala: cada iteración 10 hormigas refuerzan 10 recorridos entre muchísimos posibles, y la colonia tarda demasiado en aprender. Para que el ACO sea útil a este tamaño haría falta lo que dejamos por fuera a propósito: búsqueda local (2-opt) y una variante que refuerce más los mejores recorridos, como MAX-MIN.

7 Conclusiones

1. **El diseño decidió si el problema se podía resolver; el lenguaje decidió cuánto había que esperar.** Con 200 000 ciudades la versión de clase necesitaría 640 GB y no corre en ningún lenguaje, mientras que la optimizada lo resolvió en los cuatro, incluido Python, con menos de 150 MB.
2. **A escala moderada, el lenguaje pesó más de lo que esperábamos.** Con 2 000 ciudades, pasar de Python a C++ aceleró 64 veces y cambiar el diseño 39 veces. Pero la ventaja del diseño crece con el tamaño y la del lenguaje no (bajó a 31 veces con 200 000 ciudades).
3. **La memoria se puede predecir antes de programar.** La cuenta de 277 bytes por ciudad acertó con un error máximo de 0.1 MB entre 500 y 1 000 000 ciudades.
4. **C++ fue el más rápido y el más liviano, con Rust prácticamente empatado** (1.23 veces el tiempo de C++ e igual memoria). Java quedó cerca en tiempo (1.55 veces) pero usó más del doble de memoria, y Python fue entre 31 y 67 veces más lento.
5. **Optimizar no costó calidad, la mejoró.** Con 2 000 ciudades la versión de clase quedó peor que el vecino más cercano (56.46 contra 42.28) y la optimizada lo superó (40.06). Con $\beta = 2$, la ruleta sobre todas las ciudades reparte la mitad de la probabilidad en saltos lejanos.
6. **A 200 000 ciudades, el ACO de clase por sí solo no alcanza.** Necesitó 271 iteraciones solo para igualar al vecino más cercano. Hace falta búsqueda local para que valga la pena a esa escala.

Limitaciones. Usamos una sola instancia por tamaño (las repeticiones cambian la semilla de las hormigas, no las ciudades). Los parámetros fueron los mismos en todas las pruebas, lo cual es justo para comparar recursos pero no es lo mejor para la calidad. No pudimos usar `clang++`. Y quedaron mejoras sin aplicar que bajarían más los recursos: guardar la feromona en `float` en vez de `double` (la mitad de memoria en esas tablas) y construir las 10 hormigas en paralelo (hasta 6 veces más rápido en los lenguajes compilados).